

## TP Flutter S3 BEAUVALLET : Utilisation de Flutter et Supabase

### 1) Problématique

Nous souhaitons savoir comment relier une application Flutter à une base de données en ligne pour afficher et modifier des données, et gérer des utilisateurs. Le sujet de base proposait d'utiliser Firebase, mais nous avons préféré adapter les travaux pratiques avec Supabase pour pouvoir travailler en SQL comme nous avons l'habitude de le faire en cours. Pour cela, nous allons écrire nos fichiers de code avec Notepad++, lancer le serveur de l'application en ligne de commande avec la console Flutter, et vérifier que tout fonctionne bien en ouvrant la page dans Google Chrome.

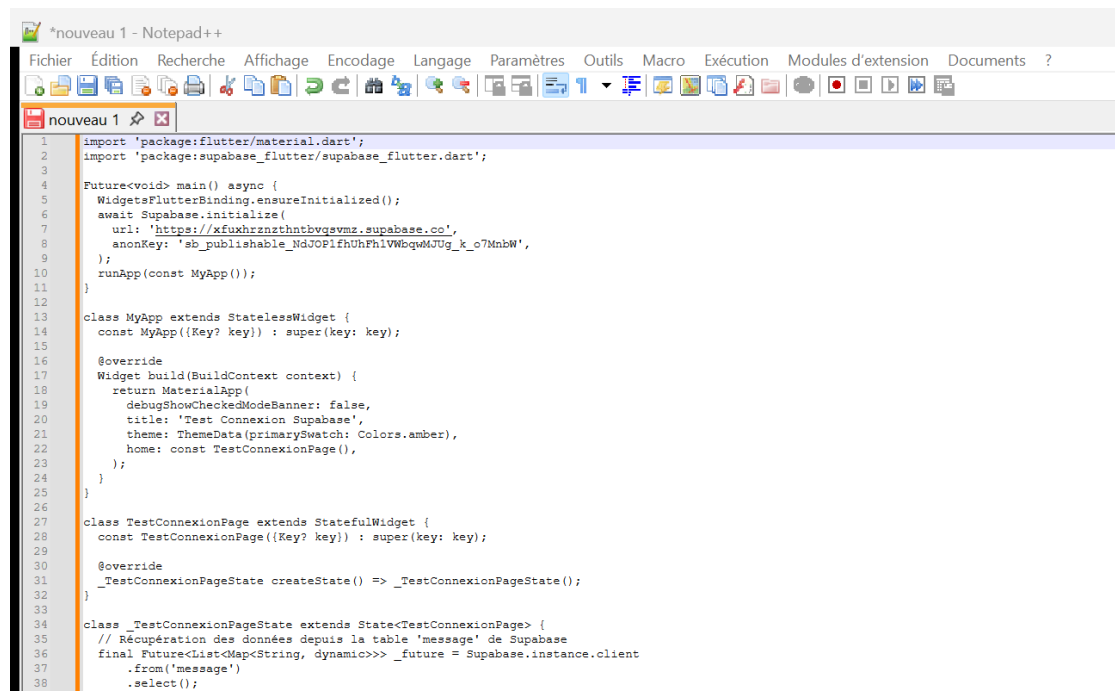
### 2) Exercices

#### Partie 1 : Les 4 tutoriels de base avec Supabase

##### Exercice 1 : Test de connexion à la base de données

Nous souhaitons d'abord tester si notre application arrive bien à se connecter à Supabase. Pour cela, on ajoute la dépendance dans pubspec.yaml, puis on écrit le code de connexion dans Notepad++ pour aller chercher le texte enregistré dans notre table message :

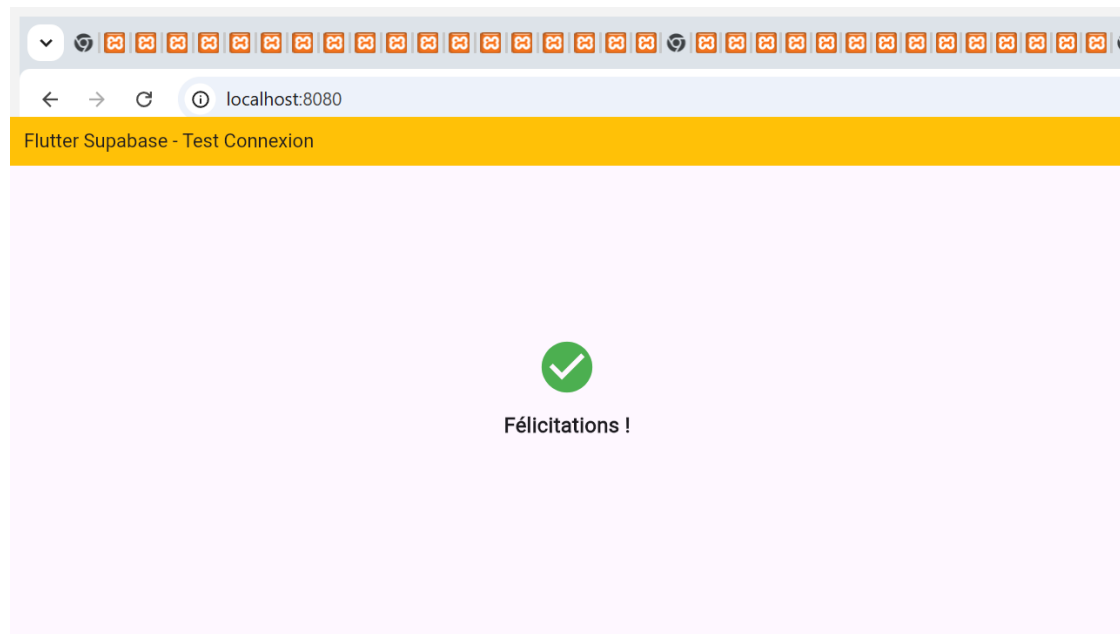
Le code ressemble à ceci :



```
1 import 'package:flutter/material.dart';
2 import 'package:supabase_flutter/supabase_flutter.dart';
3
4 Future<void> main() async {
5   WidgetsFlutterBinding.ensureInitialized();
6   await Supabase.initialize(
7     url: 'https://xfuxhrzrztthntbvqsvmz.supabase.co',
8     anonKey: 'sb_publishable_Nd3OPifhUuFh1VWbqWdMJUg_k_o7MnbW',
9   );
10  runApp(const MyApp());
11 }
12
13 class MyApp extends StatelessWidget {
14   const MyApp({Key? key}) : super(key: key);
15
16   @override
17   Widget build(BuildContext context) {
18     return MaterialApp(
19       debugShowCheckedModeBanner: false,
20       title: 'Test Connexion Supabase',
21       theme: ThemeData(primarySwatch: Colors.amber),
22       home: const TestConnexionPage(),
23     );
24   }
25 }
26
27 class TestConnexionPage extends StatefulWidget {
28   const TestConnexionPage({Key? key}) : super(key: key);
29
30   @override
31   _TestConnexionPageState createState() => _TestConnexionPageState();
32 }
33
34 class _TestConnexionPageState extends State<TestConnexionPage> {
35   // Récupération des données depuis la table 'message' de Supabase
36   final Future<List<Map<String, dynamic>>> _future = Supabase.instance.client
37     .from('message')
38     .select();
39 }
```

Comme Notepad++ est juste un éditeur de texte et ne peut pas lancer d'application tout seul, on ouvre une console pour démarrer notre projet avec la commande flutter run -d web-server --web-

port 8080. On va ensuite sur Google Chrome à l'adresse <http://localhost:8080> et on peut voir que le message de test s'affiche bien :



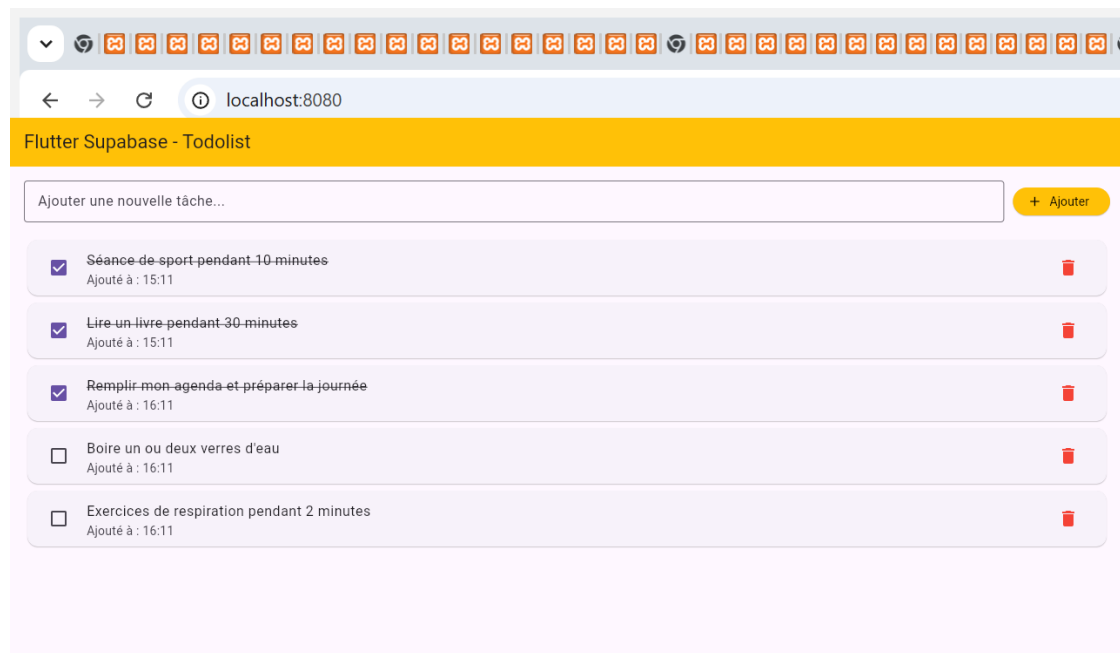
## Exercice 2 : Création de la Todolist en temps réel

Nous allons maintenant créer une liste de tâches reliée à une table items dans la base. On écrit le code dans Notepad++ pour pouvoir ajouter une tâche avec l'heure, la cocher quand elle est terminée et la supprimer avec une icône de poubelle :

Le code ressemble à ceci :

```
*nouveau 1 - Notepad++
Fichier  Édition  Recherche  Affichage  Encodage  Langage  Paramètres  Outils  Macro  Exécution  Modules d'extension  Documents  ?
nouveau 1
1  import 'package:flutter/material.dart';
2  import 'package:supabase_flutter/supabase_flutter.dart';
3  import 'package:intl/intl.dart';
4
5  Future<void> main() async {
6    WidgetsFlutterBinding.ensureInitialized();
7    await Supabase.initialize(
8      url: 'https://xfuxhrnztthrtbvgsvmz.supabase.co',
9      anonKey: 'sb_publishable_NdJOPifhUuPhlVWbqmMJUg_k_c7MnbW',
10    );
11    runApp(const MyApp());
12  }
13
14  class MyApp extends StatelessWidget {
15    const MyApp({Key? key}) : super(key: key);
16
17    @override
18    Widget build(BuildContext context) {
19      return MaterialApp(
20        debugShowCheckedModeBanner: false,
21        title: 'Todolist Supabase',
22        theme: ThemeData(primarySwatch: Colors.amber),
23        home: const TodolistPage(),
24      );
25    }
26  }
27
28  class TodolistPage extends StatefulWidget {
29    const TodolistPage({Key? key}) : super(key: key);
30
31    @override
32    _TodolistPageState createState() => _TodolistPageState();
33  }
34
35  class _TodolistPageState extends State<TodolistPage> {
36    final taskController = TextEditingController();
37    final supabase = Supabase.instance.client;
38  }
```

Sur Google Chrome, on peut tester en ajoutant des tâches et en cochant les cases. On voit que la liste s'actualise directement :



### Exercice 3 : Inscription, connexion et page de profil

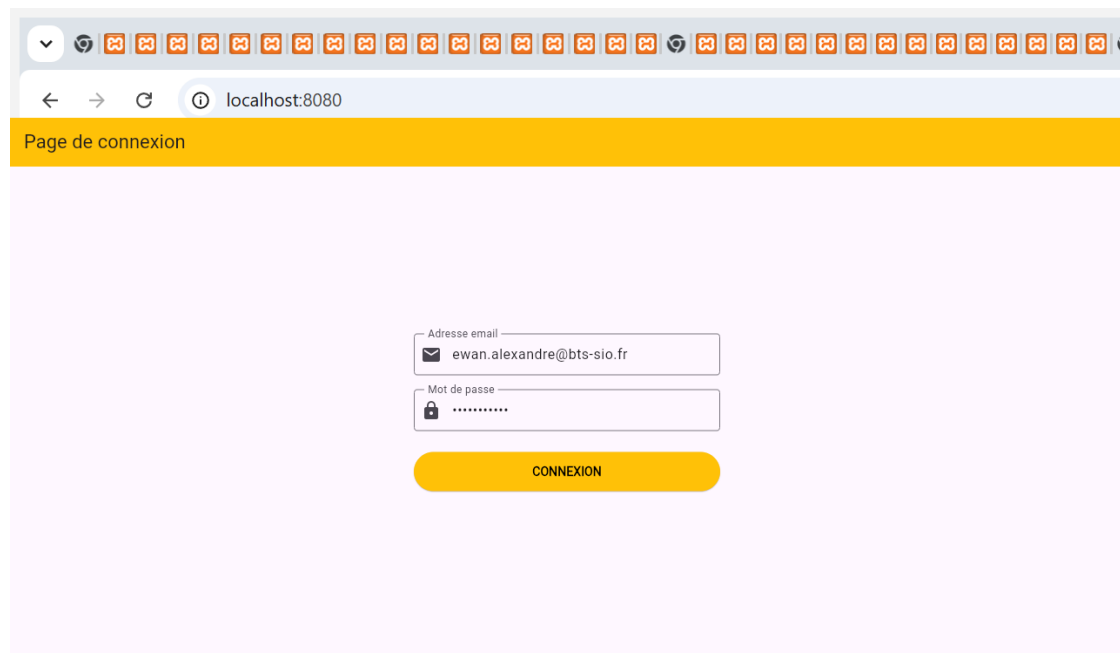
Nous voulons maintenant ajouter un système de connexion pour que l'utilisateur puisse taper son email et son mot de passe, puis être redirigé vers sa page de profil personnel. Voici le code tapé dans Notepad++ :

Le code ressemble à ceci :

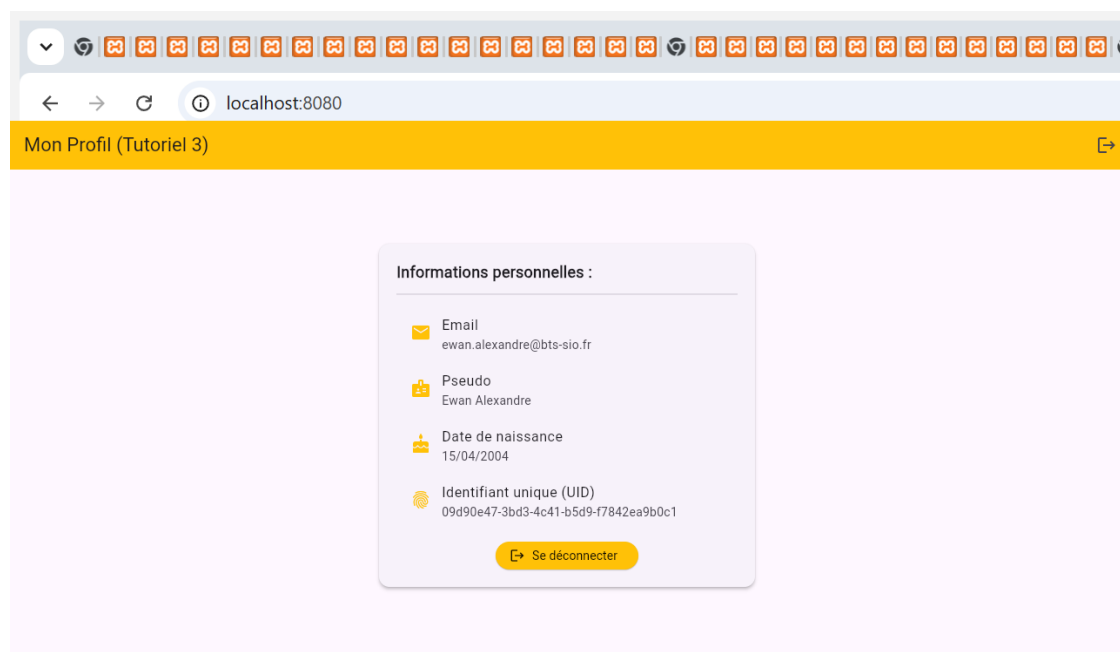
```
*nouveau 1 - Notepad++
Fichier  Édition  Recherche  Affichage  Encodage  Langage  Paramètres  Outils  Macro  Exécution  Modules d'extension  Documents  ?

nouveau 1
1 import 'package:flutter/material.dart';
2 import 'package:supabase_flutter/supabase_flutter.dart';
3
4 Future<void> main() async {
5   WidgetsFlutterBinding.ensureInitialized();
6   await Supabase.initialize(
7     url: 'https://xfuxhrznzthntbvqsvmz.supabase.co',
8     anonKey: 'sb_publishable_NdJOPifhUhFh1VWbqMwJUG_k_o7MnbW',
9   );
10  runApp(const MyApp());
11 }
12
13 class MyApp extends StatelessWidget {
14   const MyApp({Key? key}) : super(key: key);
15
16   @override
17   Widget build(BuildContext context) {
18     return MaterialApp(
19       debugShowCheckedModeBanner: false,
20       title: 'Auth & Profil Simple',
21       theme: ThemeData(primarySwatch: Colors.amber),
22       home: StreamBuilder<AuthState>({
23         stream: Supabase.instance.client.auth.onAuthStateChange,
24         builder: (context, snapshot) {
25           final session = Supabase.instance.client.auth.currentSession;
26           if (session == null) {
27             return const LoginPage();
28           } else {
29             return const SimpleProfilePage();
30           }
31         },
32       ),
33     );
34   }
35 }
36
37 class LoginPage extends StatefulWidget {
38   const LoginPage({Key? key}) : super(key: key);
39 }
```

Quand on ouvre la page sur Google Chrome sans être connecté, l'application affiche le formulaire de connexion :



Dès qu'on valide notre compte, on arrive sur la page de profil qui affiche notre email, notre identifiant et nos informations :



#### Exercice 4 : Stockage de la photo de profil et informations modifiables

Pour ce quatrième exercice, nous adaptons le stockage de fichiers en utilisant le bucket avatars de Supabase Storage. On permet à l'utilisateur de charger une photo de profil arrondie avec CircleAvatar, et de renseigner son pseudo, sa ville et sa biographie. Voici le code dans Notepad++ :  
Le code ressemble à ceci :

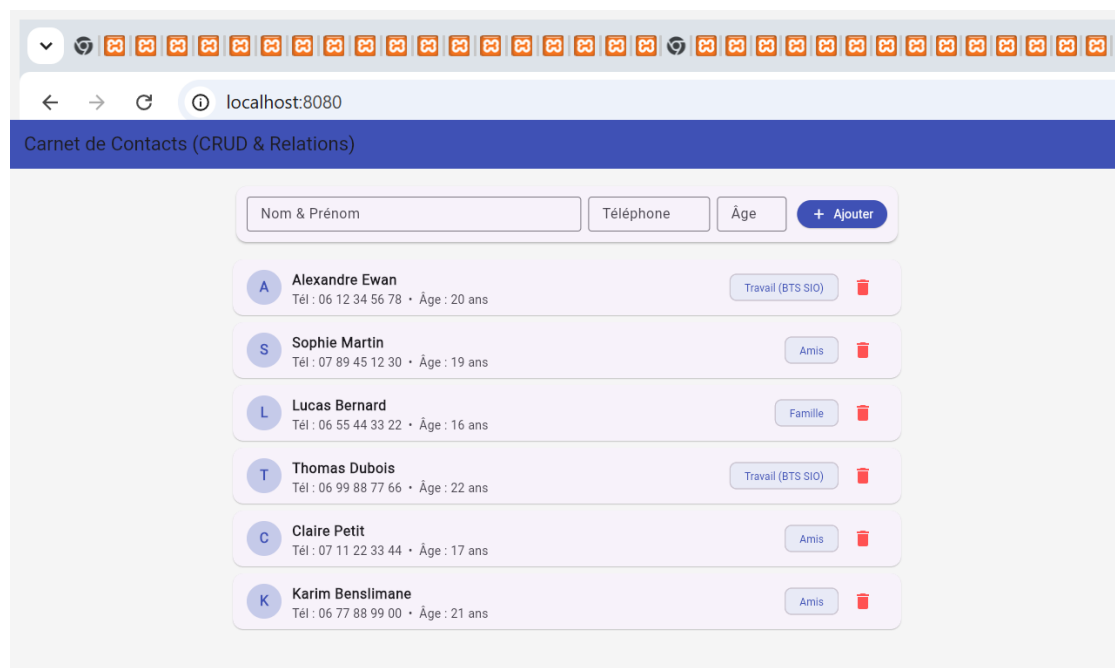


```

1 import 'package:flutter/material.dart';
2 import 'package:supabase_flutter/supabase_flutter.dart';
3
4 Future<void> main() async {
5   WidgetsFlutterBinding.ensureInitialized();
6   await Supabase.initialize(
7     url: 'https://xfuxhrznzthntbvgsvmz.supabase.co',
8     anonKey: 'sb_publishable_NdJOP1fHUhFh1VWbqWJUG_k_o7MnbW',
9   );
10  runApp(const MyApp());
11 }
12
13 class MyApp extends StatelessWidget {
14   const MyApp({Key? key}) : super(key: key);
15
16   @override
17   Widget build(BuildContext context) {
18     return MaterialApp(
19       debugShowCheckedModeBanner: false,
20       title: 'Carnet de Contacts',
21       theme: ThemeData(primarySwatch: Colors.indigo),
22       home: const ContactsPage(),
23     );
24   }
25 }
26
27 class ContactsPage extends StatefulWidget {
28   const ContactsPage({Key? key}) : super(key: key);
29
30   @override
31   _ContactsPageState createState() => _ContactsPageState();
32 }
33
34 class _ContactsPageState extends State<ContactsPage> {
35   final _nomController = TextEditingController();
36   final _telController = TextEditingController();
37   final _ageController = TextEditingController();
38   List<dynamic> _contacts = [];
39   List<dynamic> _groups = [];
40 }

```

Quand on va sur Google Chrome, la liste de nos contacts s'affiche bien avec le badge de leur groupe (Travail BTS SIO, Amis, Famille) et les boutons d'action :



### Niveaux 3 et 4 : Sécurité RLS et affichage en temps réel avec .stream()

Pour le niveau 3, nous activons la sécurité RLS (Row Level Security) sur la table contacts pour que chaque utilisateur connecté ne voie que ses propres contacts. Pour le niveau 4, on utilise la fonction .stream() pour écouter la base en continu. Voici le code dans Notepad++ :

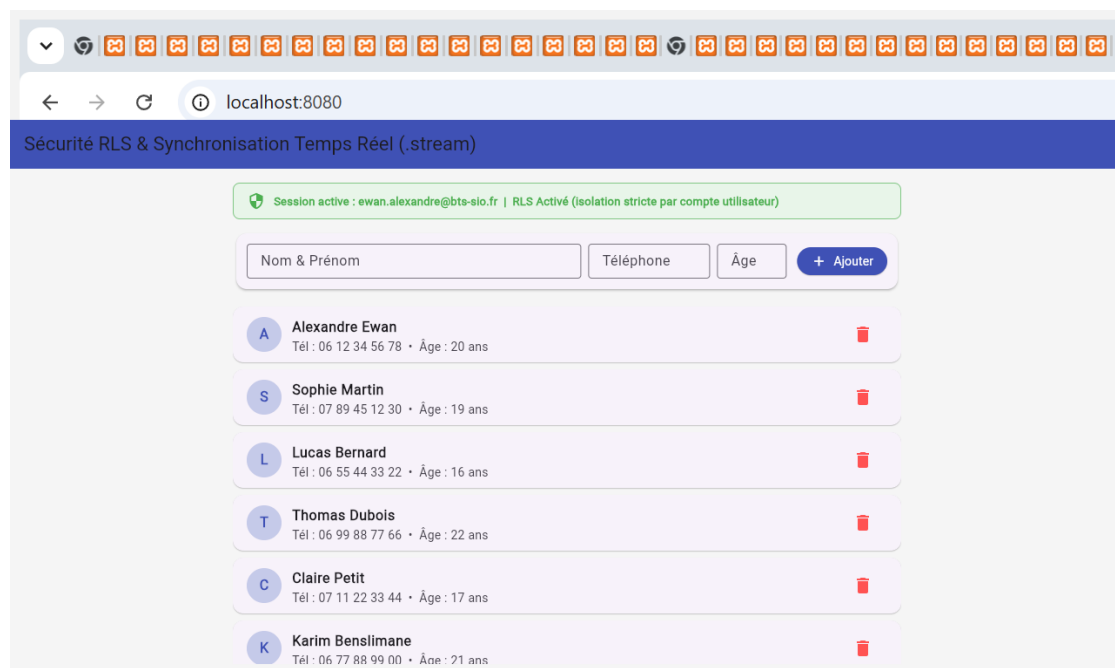
Le code ressemble à ceci :

```

1 import 'package:flutter/material.dart';
2 import 'package:supabase_flutter/supabase_flutter.dart';
3
4 Future<void> main() async {
5   WidgetsFlutterBinding.ensureInitialized();
6   await Supabase.initialize(
7     url: 'https://xfuxhrznzthntbvgsvmz.supabase.co',
8     anonKey: 'sb_publishable_NdJOP1fHUhFh1VWbqWJUG_k_o7MnbW',
9   );
10  runApp(const MyApp());
11 }
12
13 class MyApp extends StatelessWidget {
14   const MyApp({Key? key}) : super(key: key);
15
16   @override
17   Widget build(BuildContext context) {
18     return MaterialApp(
19       debugShowCheckedModeBanner: false,
20       title: 'Sécurité RLS & Stream Temps Réel',
21       theme: ThemeData(primarySwatch: Colors.indigo),
22       home: const ContactsRlsStreamPage(),
23     );
24   }
25 }
26
27 class ContactsRlsStreamPage extends StatefulWidget {
28   const ContactsRlsStreamPage({Key? key}) : super(key: key);
29
30   @override
31   _ContactsRlsStreamPageState createState() => _ContactsRlsStreamPageState();
32 }
33
34 class _ContactsRlsStreamPageState extends State<ContactsRlsStreamPage> {
35   final _nomController = TextEditingController();
36   final _telController = TextEditingController();
37   final _ageController = TextEditingController();
38   String _userEmail = 'ewan.alexandre@bts-sio.fr';
39 }

```

Sur Google Chrome, on voit le bandeau vert indiquant que la sécurité RLS est bien active, et lorsqu'un nouveau contact est ajouté, il apparaît tout de suite sur l'écran sans rafraîchir la page :



## Niveau 5 : Calcul de statistiques avec une fonction backend

Pour le niveau 5, plutôt que de faire les calculs dans l'application sur le téléphone, on crée une fonction SQL `calculer_statistiques()` directement sur Supabase qui compte le nombre total de contacts et le nombre de majeurs (âge supérieur ou égal à 18 ans). On l'appelle ensuite avec `supabase.rpc()`. Voici le code dans Notepad++ :

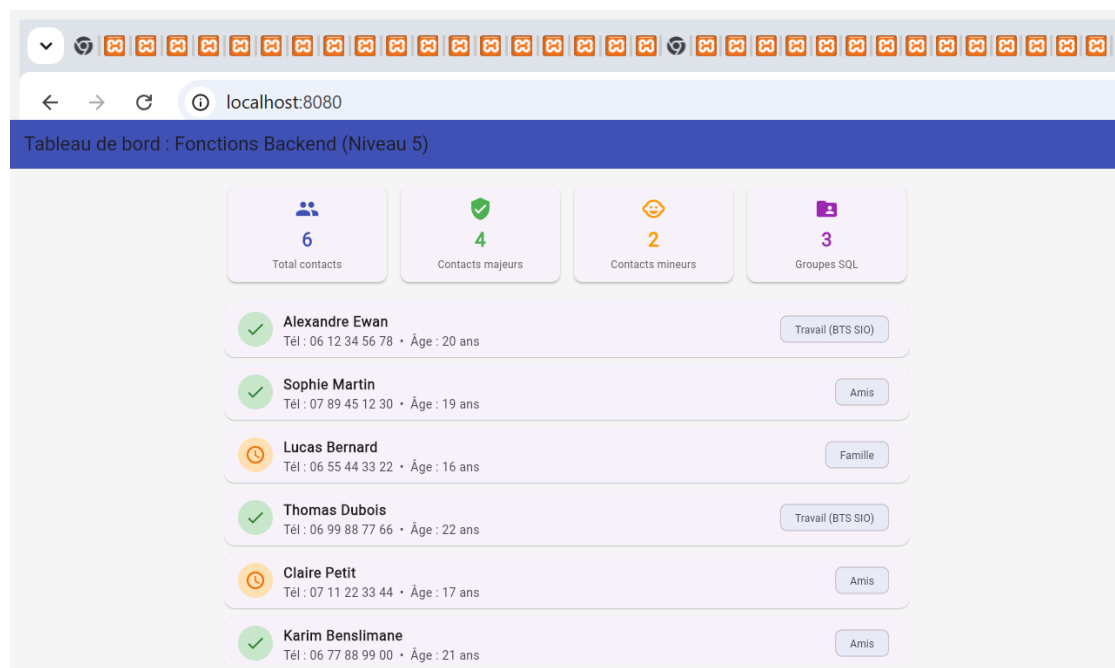
Le code ressemble à ceci :

```

1 import 'package:flutter/material.dart';
2 import 'package:supabase_flutter/supabase_flutter.dart';
3
4 Future<void> main() async {
5   WidgetsFlutterBinding.ensureInitialized();
6   await Supabase.initialize(
7     url: 'https://xfuxhrznztthtbtvgvmz.supabase.co',
8     anonKey: 'sb_publishable_NdJOP1fHUhFh1VWbqWJUG_k_o7MnbW',
9   );
10  runApp(const MyApp());
11 }
12
13 class MyApp extends StatelessWidget {
14   const MyApp({Key? key}) : super(key: key);
15
16   @override
17   Widget build(BuildContext context) {
18     return MaterialApp(
19       debugShowCheckedModeBanner: false,
20       title: 'Statistiques Backend Supabase',
21       theme: ThemeData(primarySwatch: Colors.indigo),
22       home: const StatsPage(),
23     );
24   }
25 }
26
27 class StatsPage extends StatefulWidget {
28   const StatsPage({Key? key}) : super(key: key);
29
30   @override
31   _StatsPageState createState() => _StatsPageState();
32 }
33
34 class _StatsPageState extends State<StatsPage> {
35   Map<String, dynamic> _stats = {
36     'total_contacts': 0,
37     'contacts_majeurs': 0,
38     'contacts_mineurs': 0,
39     'groupes_sql': 0,
40   };
41 }

```

Sur Google Chrome, on peut voir la carte de statistiques au-dessus de la liste avec les valeurs calculées directement par le serveur :



### 3) Conclusion

En conclusion, ce TP m'a permis d'apprendre à relier une application Flutter à une base de données en ligne comme Supabase. J'ai pu voir comment afficher des données, gérer l'inscription et la connexion d'un utilisateur, stocker des images pour un profil, et faire un carnet de contacts avec des relations SQL et du temps réel. J'ai aussi appris comment lancer le projet depuis la console et vérifier le résultat sur Google Chrome tout en écrivant le code dans Notepad++.